

UNIT-4 DATA FILTERING AND CLEANING

1. Data Filtering and Cleaning

Data filtering and cleaning are essential steps in preparing data for analysis. Filtering helps in selecting specific rows and columns of interest, while cleaning ensures that the data is free from errors, inconsistencies, and missing values. **The `dplyr` package in R provides a comprehensive set of functions to facilitate these processes.**

Key Functions and Examples:

`filter()`: Used to filter rows based on specific conditions.

`select()`: Used to select specific columns.

`mutate()`: Used to add or modify columns.

`arrange()`: Used to sort data by columns.

library(dplyr)

Sample data

```
data <- data.frame(  
  name = c("Alice", "Bob", "Charlie", "David"),  
  age = c(25, 30, 35, 40),  
  salary = c(50000, 60000, 70000, 80000)  
)
```

Filtering rows where age is greater than 30

```
filtered_data <- data %>% filter(age > 30)
```

Selecting specific columns

```
selected_data <- data %>% select(name, age)
```

Adding a new column with mutated data

```
data <- data %>% mutate(salary_in_thousands = salary / 1000)
```

UNIT-4 DATA FILTERING AND CLEANING

Arranging data by age

```
arranged_data <- data %>% arrange(age)
```

2. Subsetting and Filtering Data

Subsetting involves selecting a portion of data based on certain conditions. This can be done for both rows and columns.

Key Functions and Examples:

``subset()``: A base R function for subsetting data.

``filter()``: For row-wise subsetting.

``select()``: For column-wise subsetting.

Base R subsetting

```
subset_data <- subset(data, age > 30)
```

dplyr subsetting

```
filtered_data <- data %>% filter(age > 30)
```

```
selected_data <- data %>% select(name, age)
```

Subsetting and filtering data are essential operations in data manipulation using R. Below are examples of how to subset and filter data using various techniques in R programming.

1. Subsetting Data

Subsetting involves selecting specific rows and columns from a data frame based on certain conditions.

Example: Subsetting by Rows and Columns

UNIT-4 DATA FILTERING AND CLEANING

Suppose you have the following data frame:

Sample data frame

```
df <- data.frame(  
  Name = c("John", "Alice", "Bob", "Carol"),  
  Age = c(28, 34, 22, 45),  
  Salary = c(50000, 60000, 45000, 70000),  
  Department = c("HR", "Finance", "IT", "Admin")  
)
```

Display the original data frame

```
print(df)
```

Output:

	Name	Age	Salary	Department
1	John	28	50000	HR
2	Alice	34	60000	Finance
3	Bob	22	45000	IT
4	Carol	45	70000	Admin

Subsetting Specific Columns:

You can subset specific columns by their names or indices.

Subset columns by name

```
df_subset_cols <- df[, c("Name", "Salary")]
```

UNIT-4 DATA FILTERING AND CLEANING

Display the subsetted data frame

```
print(df_subset_cols)
```

Output:

```
   Name Salary
1  John  50000
2  Alice 60000
3   Bob  45000
4  Carol 70000
```

Subsetting Specific Rows:

You can subset specific rows based on row indices.

Subset rows by index

```
df_subset_rows <- df[1:2, ]
```

Display the subsetted data frame

```
print(df_subset_rows)
```

Output:

```
   Name Age Salary Department
1  John  28  50000         HR
2  Alice 34 60000    Finance
```

Subsetting Both Rows and Columns:

UNIT-4 DATA FILTERING AND CLEANING

You can combine row and column subsetting.

Subset specific rows and columns

```
df_subset <- df[1:3, c("Name", "Age")]
```

Display the subsetted data frame

```
print(df_subset)
```

Output:

```
  Name Age
1 John  28
2 Alice 34
3  Bob  22
```

2. Filtering Data

Filtering involves selecting rows that meet specific conditions.

#Example: Filtering Rows Based on a Condition

You can filter rows where a specific column meets a condition, such as selecting employees with a salary greater than 50,000.

Filter rows where Salary is greater than 50,000

```
df_filtered <- df[df$Salary > 50000, ]
```

Display the filtered data frame

```
print(df_filtered)
```

Output:

```
  Name Age Salary Department
2 Alice 34  60000  Finance
```

UNIT-4 DATA FILTERING AND CLEANING

```
4 Carol 45 70000 Admin
```

#Example: Filtering with Multiple Conditions

You can also filter data based on multiple conditions, such as selecting employees from the IT department with a salary less than 60,000.

Filter rows where Department is 'IT' and Salary is less than 60,000

```
df_filtered_multiple <- df[df$Department == "IT" & df$Salary < 60000, ]
```

Display the filtered data frame

```
print(df_filtered_multiple)
```

Output:

```
  Name Age Salary Department
3  Bob  22 45000         IT
```

3. Using `dplyr` for Subsetting and Filtering

The `dplyr` package provides more intuitive functions for subsetting and filtering.

#Example: Using `select()` to Subset Columns

Install and load dplyr package

```
install.packages("dplyr")
```

```
library(dplyr)
```

Subset specific columns using select()

```
df_select <- select(df, Name, Salary)
```

Display the subsetted data frame

```
print(df_select)
```

UNIT-4 DATA FILTERING AND CLEANING

Output:

```
Name Salary
1 John 50000
2 Alice 60000
3 Bob 45000
4 Carol 70000
```

#Example: Using `filter()` to Filter Rows

Filter rows where Age is greater than 30 using filter()

```
df_filter <- filter(df, Age > 30)
```

Display the filtered data frame

```
print(df_filter)
```

Output:

```
Name Age Salary Department
2 Alice 34 60000 Finance
4 Carol 45 70000 Admin
```

3. Adding, Removing, and Renaming Variables/Columns

Adding, removing, and renaming columns are common operations during data manipulation.

Key Functions and Examples:

``mutate()``: For adding or modifying columns.

``select()``: With the ``-`` sign for removing columns.

``rename()``: For renaming columns.

UNIT-4 DATA FILTERING AND CLEANING

Adding a new column

```
data <- data %>% mutate(new_column = age + 10)
```

Removing a column

```
data <- data %>% select(-salary)
```

Renaming a column

```
data <- data %>% rename(new_age = age)
```

Here are examples of adding, removing, and renaming variables (columns) in a data frame using R programming:

1. Adding Variables/Columns

You can add a new column to a data frame by simply assigning a vector of values to a new column name.

#Example: Adding a New Column

Sample data frame

```
df <- data.frame(  
  Name = c("John", "Alice", "Bob", "Carol"),  
  Age = c(28, 34, 22, 45),  
  Salary = c(50000, 60000, 45000, 70000)  
)
```

Add a new column 'Department'

```
df$Department <- c("HR", "Finance", "IT", "Admin")
```

Display the updated data frame

```
print(df)
```


UNIT-4 DATA FILTERING AND CLEANING

Output:

	Name	Age	Salary	Department
1	John	28	50000	HR
2	Alice	34	60000	Finance
3	Bob	22	45000	IT
4	Carol	45	70000	Admin

2. Removing Variables/Columns

To remove a column, you can either set it to `NULL` or use the `select()` function from the `dplyr` package.

#Example: Removing a Column Using `NULL`

Remove the 'Salary' column by setting it to NULL

```
df$Salary <- NULL
```

Display the updated data frame

```
print(df)
```

Output:

	Name	Age	Department
1	John	28	HR
2	Alice	34	Finance
3	Bob	22	IT
4	Carol	45	Admin

#Example: Removing a Column Using `dplyr`

UNIT-4 DATA FILTERING AND CLEANING

Install and load dplyr package

```
install.packages("dplyr")
```

```
library(dplyr)
```

Remove the 'Age' column using select() function

```
df <- select(df, -Age)
```

Display the updated data frame

```
print(df)
```

Output:

```
  Name Department
1  John      HR
2 Alice  Finance
3   Bob      IT
4 Carol   Admin
```

3. Renaming Variables/Columns

Renaming columns can be done using the `names()` function or the `rename()` function from the `dplyr` package.

#Example: Renaming Columns Using `names()`

Rename the 'Department' column to 'Dept'

```
names(df)[names(df) == "Department"] <- "Dept"
```

Display the updated data frame

```
print(df)
```

UNIT-4 DATA FILTERING AND CLEANING

Output:

```
  Name Dept
1 John  HR
2 Alice Finance
3 Bob   IT
4 Carol Admin
```

#Example: Renaming Columns Using `dplyr`

Rename the 'Dept' column back to 'Department' using rename() function

```
df <- rename(df, Department = Dept)
```

Display the updated data frame

```
print(df)
```

Output:

```
  Name Department
1 John      HR
2 Alice  Finance
3 Bob       IT
4 Carol   Admin
```

4. Data Cleaning and Transformation

Data cleaning involves correcting or removing incorrect, incomplete, or irrelevant data.

Transformation involves converting data into a suitable format for analysis.

Key Functions and Examples:

UNIT-4 DATA FILTERING AND CLEANING

``mutate()``: For data transformation.

``na.omit()``: For removing rows with missing values.

``replace()``: For replacing specific values in the data.

Data transformation

```
data <- data %>% mutate(salary_in_thousands = salary / 1000)
```

Removing rows with missing values

```
cleaned_data <- na.omit(data)
```

Replacing specific values

```
data$salary[data$salary < 0] <- NA
```

Data cleaning and transformation are essential steps in preparing data for analysis in R. Below are examples demonstrating different aspects of data cleaning and transformation using R programming.

1. Handling Missing Values

Handling missing values is crucial in data cleaning. You can identify, remove, or impute missing values.

#Example: Identifying Missing Values

Sample data frame with missing values

```
df <- data.frame(  
  Name = c("John", "Alice", "Bob", "Carol", NA),  
  Age = c(28, 34, NA, 45, 30),  
  Salary = c(50000, NA, 45000, 70000, 60000)  
)
```

UNIT-4 DATA FILTERING AND CLEANING

Check for missing values

```
missing_values <- is.na(df)
```

```
print(missing_values)
```

Summary of missing values

```
summary(df)
```

Output:

	Name	Age	Salary
1	FALSE	FALSE	FALSE
2	FALSE	FALSE	TRUE
3	FALSE	TRUE	FALSE
4	FALSE	FALSE	FALSE
5	TRUE	FALSE	FALSE

	Name	Age	Salary
John :1	Min. :28.00	Min. :45000	
Alice:1	1st Qu.:28.50	1st Qu.:50000	
Bob :1	Median :30.00	Median :55000	
Carol:1	Mean :34.25	Mean :56250	
NA's :1	3rd Qu.:36.50	3rd Qu.:61250	
	Max. :45.00	Max. :70000	
	NA's :1	NA's :1	

UNIT-4 DATA FILTERING AND CLEANING

#Example: Removing Rows with Missing Values

Remove rows with any missing values

```
df_clean <- na.omit(df)
```

Display the cleaned data frame

```
print(df_clean)
```

Output:

```
  Name Age Salary
1 John  28 50000
4 Carol 45 70000
```

#Example: Imputing Missing Values

Impute missing values with the mean of the respective column

```
df$Age[is.na(df$Age)] <- mean(df$Age, na.rm = TRUE)
```

```
df$Salary[is.na(df$Salary)] <- mean(df$Salary, na.rm = TRUE)
```

Display the data frame after imputation

```
print(df)
```

Output:

```
  Name   Age  Salary
1 John 28.00000 50000.0
2 Alice 34.00000 56250.0
3 Bob  34.25000 45000.0
```

UNIT-4 DATA FILTERING AND CLEANING

4 Carol 45.00000 70000.0

5 <NA> 30.00000 60000.0

2. Data Type Conversion

Sometimes, data types need to be converted for proper analysis, such as converting a factor to a numeric type.

#Example: Converting Factors to Numeric

Sample data frame with a factor column

```
df <- data.frame(  
  Category = factor(c("Low", "Medium", "High", "Medium", "Low")),  
  Score = c(1, 2, 3, 2, 1)  
)
```

Convert the 'Category' column from factor to numeric

```
df$Category_numeric <- as.numeric(df$Category)
```

Display the updated data frame

```
print(df)
```

Output:

	Category	Score	Category_numeric
1	Low	1	1
2	Medium	2	2
3	High	3	3
4	Medium	2	2
5	Low	1	1

UNIT-4 DATA FILTERING AND CLEANING

3. Removing Duplicates

Duplicate rows can be removed to ensure each observation is unique.

#Example: Removing Duplicate Rows

Sample data frame with duplicate rows

```
df <- data.frame(  
  Name = c("John", "Alice", "Bob", "Carol", "Alice"),  
  Age = c(28, 34, 22, 45, 34),  
  Salary = c(50000, 60000, 45000, 70000, 60000)  
)
```

Remove duplicate rows based on all columns

```
df_unique <- df[!duplicated(df), ]
```

Display the unique data frame

```
print(df_unique)
```

Output:

	Name	Age	Salary
1	John	28	50000
2	Alice	34	60000
3	Bob	22	45000
4	Carol	45	70000

4. Transforming Data

UNIT-4 DATA FILTERING AND CLEANING

Data transformation may involve scaling, normalizing, or creating new features from existing data.

#Example: Scaling and Normalizing Data

Sample data frame

```
df <- data.frame(  
  Height = c(150, 160, 170, 180, 190),  
  Weight = c(55, 65, 75, 85, 95)  
)
```

Normalize the 'Height' and 'Weight' columns

```
df$Height_scaled <- scale(df$Height)  
df$Weight_scaled <- scale(df$Weight)
```

Display the transformed data frame

```
print(df)
```

Output:

	Height	Weight	Height_scaled	Weight_scaled
1	150	55	-1.414214	-1.414214
2	160	65	-0.707107	-0.707107
3	170	75	0.000000	0.000000
4	180	85	0.707107	0.707107
5	190	95	1.414214	1.414214

5. Identifying and Handling Missing Values

UNIT-4 DATA FILTERING AND CLEANING

Handling missing values is crucial for accurate analysis. Missing values can be identified and handled through various methods.

Key Functions and Examples:

``is.na()``: To check for NA values.

``sum(is.na())``: To count the number of NA values.

``na.omit()``: To remove rows with NA values.

``replace()``: To replace NA values with specific values.

Identifying missing values

```
missing_values <- sum(is.na(data))
```

Removing rows with missing values

```
cleaned_data <- na.omit(data)
```

Replacing missing values with the mean of the column

```
data$salary[is.na(data$salary)] <- mean(data$salary, na.rm = TRUE)
```

Handling missing values is a common task in data cleaning. Below are examples demonstrating how to identify and handle missing values in R using various methods.

1. Identifying Missing Values

#Example: Checking for Missing Values

Sample data frame with missing values

```
df <- data.frame(  
  
  Name = c("John", "Alice", "Bob", "Carol", NA),  
  
  Age = c(28, 34, NA, 45, 30),  
  
  Salary = c(50000, NA, 45000, 70000, 60000)  
)
```

UNIT-4 DATA FILTERING AND CLEANING

Check for missing values

```
missing_values <- is.na(df)
print(missing_values)
```

Output:

	Name	Age	Salary
1	FALSE	FALSE	FALSE
2	FALSE	FALSE	TRUE
3	FALSE	TRUE	FALSE
4	FALSE	FALSE	FALSE
5	TRUE	FALSE	FALSE

#Example: Summary of Missing Values

Summary of missing values in each column

```
summary(df)
```

Output:

	Name	Age	Salary
John :1	Min. :28.00	Min. :45000	
Alice:1	1st Qu.:28.50	1st Qu.:50000	
Bob :1	Median :30.00	Median :55000	
Carol:1	Mean :34.25	Mean :56250	
NA's :1	3rd Qu.:36.50	3rd Qu.:61250	

UNIT-4 DATA FILTERING AND CLEANING

Max. :45.00 Max. :70000

NA's :1 NA's :1

#Example: Counting Missing Values in Each Column

Count the number of missing values in each column

```
missing_counts <- colSums(is.na(df))
```

```
print(missing_counts)
```

Output:

	Name	Age	Salary
--	------	-----	--------

1		1	1
---	--	---	---

2. Handling Missing Values

#Example: Removing Rows with Missing Values

Remove rows with any missing values

```
df_clean <- na.omit(df)
```

Display the cleaned data frame

```
print(df_clean)
```

Output:

	Name	Age	Salary
--	------	-----	--------

1	John	28	50000
---	------	----	-------

4	Carol	45	70000
---	-------	----	-------

UNIT-4 DATA FILTERING AND CLEANING

#Example: Imputing Missing Values with Mean

Impute missing values in the 'Age' and 'Salary' columns with the mean

```
df$Age[is.na(df$Age)] <- mean(df$Age, na.rm = TRUE)
df$Salary[is.na(df$Salary)] <- mean(df$Salary, na.rm = TRUE)
```

Display the data frame after imputation

```
print(df)
```

Output:

```
  Name Age Salary
1 John  28 50000
2 Alice 34 56250
3 Bob   34 45000
4 Carol 45 70000
5 <NA> 30 60000
```

#Example: Imputing Missing Values with Median

Impute missing values in the 'Age' and 'Salary' columns with the median

```
df$Age[is.na(df$Age)] <- median(df$Age, na.rm = TRUE)
df$Salary[is.na(df$Salary)] <- median(df$Salary, na.rm = TRUE)
```

Display the data frame after imputation

```
print(df)
```

UNIT-4 DATA FILTERING AND CLEANING

Output:

```
Name Age Salary
1 John 28 50000
2 Alice 34 56250
3 Bob 34 45000
4 Carol 45 70000
5 <NA> 30 56250
```

#Example: Filling Missing Values Forward (Using the Last Observation)

Sample data frame with missing values

```
df<- data.frame(
  Day = 1:5,
  Temperature = c(22, 23, NA, 25, NA)
)
```

Fill missing values forward

```
df$Temperature <- zoo::na.locf(df$Temperature)
```

Display the data frame after filling missing values forward

```
print(df)
```

Output:

```
Day Temperature
1 1           22
2 2           23
```

UNIT-4 DATA FILTERING AND CLEANING

3	3	23
4	4	25
5	5	25

#Example: Filling Missing Values Backward (Using the Next Observation)

Fill missing values backward

```
df$Temperature <- zoo::na.locf(df$Temperature, fromLast = TRUE)
```

Display the data frame after filling missing values backward

```
print(df)
```

Output:

Day Temperature		
1	1	22
2	2	23
3	3	25
4	4	25
5	5	NA

6. Data Type Conversion and Recoding Variables

Converting data types and recoding variables are important for ensuring the data is in the correct format for analysis.

Key Functions and Examples:

``as.numeric()`, `as.character()`, `as.factor()``: For data type conversion.

``mutate()``: With conditional logic for recoding variables.

UNIT-4 DATA FILTERING AND CLEANING

`cut()`: For binning continuous variables into categories.

Data type conversion

```
data$age <- as.numeric(data$age)
```

```
data$name <- as.character(data$name)
```

Recoding variables using mutate and case_when

```
data <- data %>% mutate(  
  age_group = case_when(  
    age < 18 ~ "Child",  
    age >= 18 & age < 65 ~ "Adult",  
    age >= 65 ~ "Senior"  
  )  
)
```

Binning continuous variable into categories

```
data <- data %>% mutate(salary_bin = cut(salary, breaks = c(0, 50000, 100000, Inf), labels =  
c("Low", "Medium", "High")))
```

Data type conversion and recoding variables are common tasks in data processing using R. Below are examples demonstrating how to perform these tasks.

1. Data Type Conversion

#Example: Converting Data Types in R

Sample data frame with different data types

```
df <- data.frame(  
  # Example data frame with different data types  
  # ...  
)
```


UNIT-4 DATA FILTERING AND CLEANING

```
ID = c(1, 2, 3, 4, 5),  
Name = c("John", "Alice", "Bob", "Carol", "Eve"),  
Age = c(28, 34, 45, 22, 30),  
Salary = c("50000", "60000", "70000", "80000", "90000")  
)
```

Check the current data types

```
str(df)
```

Output:

```
'data.frame':  5 obs. of  4 variables:
```

```
$ ID      : num  1 2 3 4 5
```

```
$ Name    : chr  "John" "Alice" "Bob" "Carol" ...
```

```
$ Age     : num  28 34 45 22 30
```

```
$ Salary  : chr  "50000" "60000" "70000" "80000" ...
```

#Example: Converting Character to Numeric

Convert the 'Salary' column from character to numeric

```
df$Salary <- as.numeric(df$Salary)
```

Check the updated data types

```
str(df)
```

Output:

```
'data.frame' :  5 obs. of  4 variables:
```

UNIT-4 DATA FILTERING AND CLEANING

```
$ ID      : num  1 2 3 4 5
$ Name    : chr  "John" "Alice" "Bob" "Carol" ...
$ Age     : num  28 34 45 22 30
$ Salary  : num  50000 60000 70000 80000 90000
```

#Example: Converting Numeric to Factor

Convert the 'Age' column from numeric to factor

```
df$Age <- as.factor(df$Age)
```

Check the updated data types

```
str(df)
```

Output:

```
'data.frame' : 5 obs. of 4 variables:
```

```
$ ID      : num  1 2 3 4 5
$ Name    : chr  "John" "Alice" "Bob" "Carol" ...
$ Age     : Factor w/ 4 levels "22","28","34",...: 2 3 4 1 2
$ Salary  : num  50000 60000 70000 80000 90000
```

2. Recoding Variables

#Example: Recoding Numeric Variables

Recoding 'Age' into age groups

```
df$AgeGroup <- cut(as.numeric(df$Age),  
breaks = c(20, 30, 40, 50),  
labels = c("20-30", "30-40", "40-50"),
```

UNIT-4 DATA FILTERING AND CLEANING

```
right = FALSE)
```

```
# Display the data frame after recoding
```

```
print(df)
```

Output:

```
ID Name Age Salary AgeGroup
1 1 John  28  50000  20-30
2 2 Alice 34  60000  30-40
3 3 Bob   45  70000  40-50
4 4 Carol 22  80000  20-30
5 5 Eve   30  90000  30-40
```

#Example: Recoding Categorical Variables

```
# Recoding 'Name' to group people by first letter
```

```
df$NameGroup <- ifelse(grepl("^[A-M]", df$Name), "A-M", "N-Z")
```

```
# Display the data frame after recoding
```

```
print(df)
```

Output:

```
ID Name Age Salary AgeGroup NameGroup
1 1 John  28  50000  20-30    A-M
2 2 Alice 34  60000  30-40    A-M
```

UNIT-4 DATA FILTERING AND CLEANING

```
3 3 Bob 45 70000 40-50 A-M
4 4 Carol 22 80000 20-30 A-M
5 5 Eve 30 90000 30-40 A-M
```

#Example: Recoding Numeric to Binary

Recoding 'Salary' into a binary variable (High: Salary > 60000, Low: Salary <= 60000)

```
df$SalaryLevel <- ifelse(df$Salary > 60000, "High", "Low")
```

Display the data frame after recoding

```
print(df)
```

Output:

ID	Name	Age	Salary	AgeGroup	NameGroup	SalaryLevel
1 1	John	28	50000	20-30	A-M	Low
2 2	Alice	34	60000	30-40	A-M	Low
3 3	Bob	45	70000	40-50	A-M	High
4 4	Carol	22	80000	20-30	A-M	High
5 5	Eve	30	90000	30-40	A-M	High

Practical Example: Cleaning and Analyzing Employee Data

Let's go through a practical example of cleaning and analyzing an employee dataset:

```
library(dplyr)
```

```
# Sample employee data
```

UNIT-4 DATA FILTERING AND CLEANING

```
employee_data <- data.frame(  
  employee_id = 1:5,  
  name = c("Alice", "Bob", "Charlie", "David", "Eva"),  
  age = c(25, 30, NA, 40, 35),  
  salary = c(50000, 60000, 70000, -1, 80000),  
  department = c("HR", "Finance", "IT", "Finance", "HR")  
)
```

Step 1: Identify and handle missing values

Count missing values

```
missing_values <- sum(is.na(employee_data))
```

Replace missing values with the mean of the column

```
employee_data$age[is.na(employee_data$age)] <- mean(employee_data$age, na.rm = TRUE)
```

Step 2: Remove incorrect values

Replace negative salary values with NA

```
employee_data$salary[employee_data$salary < 0] <- NA
```

Step 3: Transform and clean data

Add a new column for salary in thousands

```
employee_data <- employee_data %>% mutate(salary_in_thousands = salary / 1000)
```

Step 4: Subset and filter data

Filter employees older than 30

```
older_employees <- employee_data %>% filter(age > 30)
```

Select specific columns

```
selected_columns <- employee_data %>% select(employee_id, name, age)
```

UNIT-4 DATA FILTERING AND CLEANING

Step 5: Data type conversion and recoding variables

Convert department to a factor

```
employee_data$department <- as.factor(employee_data$department)
```

Recoding age group

```
employee_data <- employee_data %>% mutate(
```

```
  age_group = case_when(
```

```
    age < 30 ~ "Young",
```

```
    age >= 30 & age < 40 ~ "Middle-aged",
```

```
    age >= 40 ~ "Senior"
```

```
  )
```

```
)
```

Final cleaned data

```
cleaned_employee_data <- employee_data
```

Display the cleaned data

```
print(cleaned_employee_data)
```

UNIT-4 DATA FILTERING AND CLEANING

UNIT 4 (SHORT QUESTIONS AND ANSWERS)

1. Data Filtering and Cleaning

Q1: What is data filtering and why is it important in data analysis?

A1: Data filtering is the process of selecting a subset of data based on specific criteria. It is important because it allows analysts to focus on relevant data, remove irrelevant or noisy data, and improve the accuracy of the analysis.

Q2: What is a common method for filtering data in R?

A2: A common method for filtering data in R is using the `dplyr` package with functions like `filter()` to select rows based on conditions.

Q3: How can you remove duplicate entries from a dataset?

A3: Duplicate entries can be removed from a dataset using the `distinct()` function in R or by using the 'REMOVE DUPLICATES' feature in Excel.

2. Subsetting and Filtering Data

Q4: What is subsetting in the context of data analysis?

A4: Subsetting refers to the process of extracting a portion of the dataset based on specific conditions. This can involve selecting certain rows, columns, or a combination of both.

Q5: How would you subset a data frame in R to include only rows where the value of a variable `age` is greater than 30?

A5: In R, you can subset a data frame using the `filter()` function from the `dplyr` package:

```
subset_data <- filter(data, age > 30)
```

Q6: What is the difference between subsetting and filtering?

A6: Subsetting generally refers to extracting a specific part of the data (e.g., certain rows or columns), whereas filtering specifically refers to selecting data based on criteria (e.g., rows where a condition is true).

UNIT-4 DATA FILTERING AND CLEANING

3. Adding, Removing, and Renaming Variables/Columns

Q7: How can you add a new column to a data frame in R?

A7: A new column can be added to a data frame in R by using the `mutate()` function from the `dplyr` package:

```
data <- mutate(data, new_column = existing_column * 2)
```

Q8: What is the purpose of renaming columns in a dataset?

A8: Renaming columns improves readability, ensures consistency, and makes the dataset easier to work with by using meaningful and standard names.

Q9: How do you remove a column from a data frame in R?

A9: You can remove a column from a data frame in R by using the `select()` function with a minus sign:

```
data <- select(data, -column_name)
```

4. Data Cleaning and Transformation

Q10: What is data transformation, and how does it differ from data cleaning?

A10: Data transformation involves changing the format, structure, or values of data to make it suitable for analysis (e.g., normalizing values). Data cleaning focuses on identifying and correcting errors or inconsistencies (e.g., removing duplicates or handling missing values).

Q11: Give an example of a common data transformation technique.

A11: A common data transformation technique is normalizing or scaling numeric variables so that they fall within a specific range (e.g., between 0 and 1).

Q12: What is the purpose of applying log transformation to data?

A12: Log transformation is applied to data to stabilize variance, make data more normally distributed, and reduce the impact of outliers.

5. Identifying and Handling Missing Values

Q13: Why is it important to handle missing values in a dataset?

UNIT-4 DATA FILTERING AND CLEANING

A13: Handling missing values is crucial because they can lead to biased results, reduce statistical power, and affect the accuracy of the analysis.

Q14: What are two common methods for handling missing values?

A14: Two common methods for handling missing values are:

1. Imputation: Replacing missing values with estimated ones (e.g., mean or median).
2. Deletion: Removing rows or columns with missing values.

Q15: How can you identify missing values in a dataset in R?

A15: Missing values can be identified in R using the `is.na()` function:

```
missing_values <- is.na(data)
```

6. Data Type Conversion and Recoding Variables

Q16: Why is data type conversion important in data analysis?

A16: Data type conversion ensures that the data is in the correct format for analysis. For example, converting character data to numeric allows for mathematical operations, while converting data to factors allows for categorical analysis.

Q17: How can you convert a character variable to a numeric variable in R?

A17: In R, you can convert a character variable to a numeric variable using the `as.numeric()` function:

```
data$numeric_var <- as.numeric(data$char_var)
```

Q18: What does it mean to recode a variable, and when might you do it?

A18: Recoding a variable involves changing the values of a variable to new ones (e.g., merging categories or changing a scale). This is often done to simplify analysis or when combining categories for statistical modeling.

Q19: Give an example of recoding a variable in R.

A19: You can recode a variable in R using the `recode()` function from the `dplyr` package:

UNIT-4 DATA FILTERING AND CLEANING

```
data$recoded_var <- recode(data$original_var, "OldValue1" = "NewValue1", "OldValue2" = "NewValue2")
```

Q20: What function in R can be used to change the data type of multiple variables simultaneously?

A20: The `mutate_at()` function in `dplyr` can be used to change the data type of multiple variables simultaneously:

```
data <- mutate_at(data, vars(var1, var2), as.numeric)
```

Data Filtering and cleaning

- **Q21: How to handle missing values in a data frame?**

- A21: Missing values can be handled using functions like `is.na()` to identify them, `na.omit()` to remove rows with missing values, or imputation techniques to fill in missing values. For example, `data <- na.omit(data)` removes rows with any missing values from the data frame "data".

2. Subsetting and filtering data

- **Q22: How to select specific rows and columns from a data frame?**

- A22: You can use square brackets `[]` to subset data. For example, `data[1:5, c("age", "gender")]` selects the first 5 rows and columns named "age" and "gender". The `dplyr` package provides more flexible subsetting functions like `select()` and `filter()`.

3. Adding, removing, and renaming variables/columns

- **Q23: How to create a new column in a data frame?**

- A23: You can create new columns using the assignment operator `<-`. For example, `data$new_column <- data$column1 * 2` creates a new column "new_column" by multiplying the values in "column1" by 2.

4. Data Cleaning and Transformation

UNIT-4 DATA FILTERING AND CLEANING

- **Q24: What is data normalization and how to perform it?**

- A24: Data normalization scales numerical features to a common range, often between 0 and 1. It's useful for algorithms sensitive to feature scales. You can use the `scale()` function for normalization. For example, `scaled_data <- scale(data)` normalizes the numerical columns in the data frame "data".

5. Identifying and handling missing values

- **Q25: How to check for missing values in a data frame?**

- A25: The `is.na()` function returns a logical vector indicating missing values. For example, `sum(is.na(data))` counts the total number of missing values in the data frame "data".

6. Data type conversion and recoding variables

- **Q26: How to convert a numeric variable to a factor?**

- A26: The `as.factor()` function converts a numeric vector to a factor. For example, `data$age_group <- as.factor(cut(data$age, breaks = c(0, 18, 30, 50, Inf), labels = c("young", "adult", "middle_aged", "senior")))` creates an age group variable based on the age column.

MCQ'S of Unit – 4

Data Filtering and cleaning

1. Data cleaning involves:

- a) Organizing data for analysis
- b) Identifying and correcting errors in data
- c) Visualizing data
- d) Building statistical models
- **Answer: b**

2. Which function is used to remove missing values from a data frame?

UNIT-4 DATA FILTERING AND CLEANING

- a) na.omit()
- b) remove_na()
- c) clean_data()
- d) drop_missing()
- **Answer: a**

Subsetting and filtering data

3. Which function from the dplyr package is used to select specific columns from a data frame?

- a) filter()
- b) select()
- c) arrange()
- d) mutate()
- **Answer: b**

4. How would you select rows where the value in the "age" column is greater than 30?

- a) data[age > 30,]
- b) filter(data, age > 30)
- c) subset(data, age > 30)
- All of the above
- **Answer: d**

Adding, removing, and renaming variables/columns

5. How can you rename a column named "old_name" to "new_name" in a data frame?

- a) rename(data, new_name = old_name)
- b) change_name(data, old_name, new_name)
- c) modify_column(data, old_name, new_name)
- d) There is no direct function for this
- **Answer: a**

6. Which function can be used to create a new column in a data frame?

- a) add_column()
- b) create_column()

UNIT-4 DATA FILTERING AND CLEANING

- c) mutate()
- d) new_column()
- **Answer: c**

Data Cleaning and Transformation

7. Data transformation involves:

- a) Changing the structure of data
- b) Visualizing data
- c) Importing data
- d) Building statistical models
- **Answer: a**

8. Which function is used to create a new variable based on existing variables in a data frame?

- a) transform()
- b) mutate()
- c) calculate()
- d) create()
- **Answer: b**

Identifying and handling missing values

9. Which function can be used to check for missing values in a data frame?

- a) is.na()
- b) find_missing()
- c) check_missing()
- d) missing_values()
- **Answer: a**

10. Imputation is a technique used to:

- a) Remove missing values
- b) Replace missing values with estimated values
- c) Identify missing values
- d) Ignore missing values

UNIT-4 DATA FILTERING AND CLEANING

- **Answer: b**

Data type conversion and recoding variables

11. How can you convert a numeric variable to a factor in R?

- a) as.factor()
- b) factorize()
- c) convert_to_factor()
- d) categorize()
- **Answer: a**

12. Which function is used to recode values of a variable in R?

- a) recode()
- b) factor()
- c) mutate()
- d) transform()
- **Answer: c**

Basic Data Structures

13. What is the difference between a vector and a list in R?

- a) Vectors can only store numeric data, while lists can store any data type.
- b) Vectors are homogeneous, while lists can be heterogeneous.
- c) Vectors are two-dimensional, while lists are one-dimensional.
- d) There is no difference between vectors and lists.
- **Answer: b**

14. Which data structure is best suited for storing data with different types of elements?

- a) Vector
- b) Matrix
- c) Data frame
- d) List
- **Answer: d**